PIPE NETWORK ANALYSIS

HARDY CROSS METHOD

Abstract

This report details the computational analysis of a 20-pipe hydraulic network using the Hardy Cross method. A Python algorithm was developed to determine the discharge in each branch for a specified inflow of **144.0 m^3/s**. The program automated the iterative process, satisfying mass balance and energy conservation laws with a convergence precision of 0.001. The study successfully resolved the complex network topology, and the final results include a complete table of flow magnitudes and directions for all system branches.

Lovekush Meena
23CE10030

HARDY CROSS PIPE NETWORK PROBLEM SOLVER

Name: Lovekush Meena

Roll No: 23CE10030

Subject: WATER RESOURCES AND GEOTECHNICAL ENGINEERING SESSIONAL – CE39204

Code(Python):-

```
import math

from collections import deque

# 1. GIVEN CONSTANTS

INFLOW = 144.0        # My specific inflow value based on 23CE10030

MAX_ITERS = 5000

ERROR_LIMIT = 0.001    # Convergence criteria

G = 9.81

PI = math.pi


# 2. INPUT DATA

# Storing pipe data: ID, Start, End, Length, Dia, Friction

# Data taken from the problem statement

raw_data = [

    (1, 'A', 'B', 300, 0.40, 0.018),

    (2, 'B', 'C', 250, 0.35, 0.020),

    (3, 'C', 'D', 200, 0.30, 0.022),

    (4, 'D', 'A', 280, 0.32, 0.021),

    (5, 'B', 'E', 220, 0.28, 0.024),

    (6, 'E', 'C', 260, 0.30, 0.023),

    (7, 'C', 'F', 240, 0.34, 0.020),

    (8, 'F', 'D', 210, 0.33, 0.021),

    (9, 'E', 'F', 230, 0.26, 0.026),

    (10, 'A', 'E', 270, 0.36, 0.019),

    (11, 'B', 'F', 290, 0.31, 0.022),

    (12, 'D', 'E', 250, 0.29, 0.024),

    (13, 'C', 'A', 310, 0.38, 0.018),

    (14, 'F', 'A', 280, 0.34, 0.020),

    (15, 'E', 'G', 260, 0.27, 0.025),

    (16, 'G', 'C', 300, 0.35, 0.020),

    (17, 'G', 'D', 320, 0.30, 0.023),
```

```
    (18, 'F', 'G', 240, 0.25, 0.028),
    (19, 'B', 'G', 310, 0.33, 0.021),
    (20, 'A', 'G', 350, 0.37, 0.019)
]

# Dictionary to hold all pipe info easier
pipes = {}
# This Adjacency list for finding loops later
adj_list = {}
print("Step 1: Calculating Resistance (R) values for each Pipes...")
for entry in raw_data:
    pid, u, v, length, dia, fric = entry
# Calculating Resistance R using formula: (8*f*L) / (g*pi^2*D^5) mentioned in Problem Handout
    numerator = 8 * fric * length
    denominator = G * (PI ** 2) * (dia ** 5)
    r_val = numerator / denominator

    # Saving everything to the dictionary
    pipes[pid] = {
        'id': pid,
        'u': u, 'v': v,
        'L': length, 'D': dia,
        'R': r_val,
        'Q': 0.0 # Initial flow is 0
    }
  # Building the graph connections
    if u not in adj_list: adj_list[u] = []
    if v not in adj_list: adj_list[v] = []

    # Storing neighbor info: (neighbor_node, pipe_id) for loop formation
    adj_list[u].append((v, pid))
    adj_list[v].append((u, pid))

# 3. INITIAL GUESS of flow in Pipes, We need to assign the Inflow to a valid path so that Continuity is satisfied.

# Using a simple BFS to find a path from A to G.
print("Step 2: Assigning initial flow to path A -> G...")

queue = ['A']
```

```
visited = {'A'}
parent_map = {} # To track the path back

path_found = False
while len(queue) > 0:
    curr = queue.pop(0) # Get first item
    if curr == 'G':
        path_found = True
        break

    # Check neighbors
    for neighbor, pid in adj_list[curr]:
        if neighbor not in visited:
            visited.add(neighbor)
            parent_map[neighbor] = curr
            queue.append(neighbor)

if not path_found:
    print("Error: Could not find path from A to G!")
else:
    # Trace the path back from G to A
    curr = 'G'
    while curr != 'A':
        prev = parent_map[curr]

        # Find the pipe connecting 'prev' and 'curr'
        for pid in pipes:
            p = pipes[pid]
            # Check if this pipe connects the two nodes
            if (p['u'] == prev and p['v'] == curr):
                # Flow matches direction
                pipes[pid]['Q'] += INFLOW
                break
            elif (p['u'] == curr and p['v'] == prev):
                # Flow is opposite
                pipes[pid]['Q'] -= INFLOW
                break
        curr = prev
    print("Initial flow assigned.")
```

```
# 4. FINDING LOOPS
# Using Spanning Tree logic to find fundamental loops automatically.
# Any pipe NOT in the spanning tree forms a loop.
print("Step 3: Finding fundamental loops required for error calcutions ...")

tree_edges = set()
visited_nodes = {'A'}
bfs_queue = ['A']
# Dictionary to store parent pointers for the tree: node -> (parent, pipe_id)
tree_parents = {}

# Building the Spanning Tree
while len(bfs_queue) > 0:
    u = bfs_queue.pop(0)

    for v, pid in adj_list[u]:
        if v not in visited_nodes:
            visited_nodes.add(v)
            tree_parents[v] = (u, pid)
            tree_edges.add(pid)
            bfs_queue.append(v)

# Now, any pipe not in 'tree_edges' is a chord and creates a loop
loops = []
sorted_ids = sorted(pipes.keys())

for pid in sorted_ids:
    if pid not in tree_edges:
        # This pipe closes a loop
        p = pipes[pid]
        u_start = p['u']
        v_start = p['v']

        # Start the loop list with this chord pipe (Direction 1)
        this_loop = [(pid, 1)]

        # Helper to get the path from a node to the root 'A'
        def get_path_to_root(node):
```

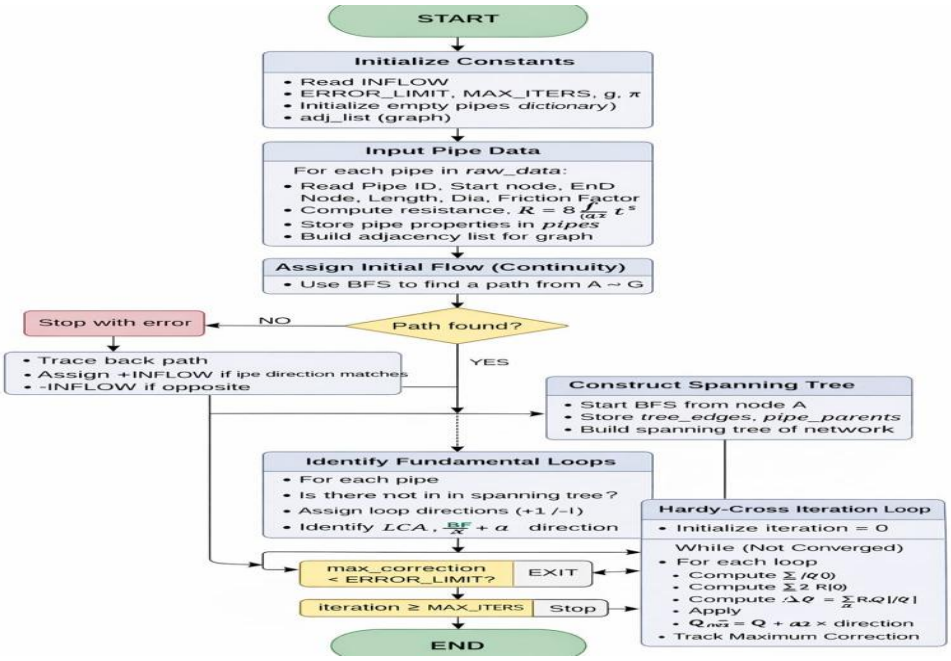
```
    path = []
    curr = node
    while curr in tree_parents:
        path.append(curr)
        curr, _ = tree_parents[curr]
    path.append('A') # Root
    return path

path_u = get_path_to_root(u_start)
path_v = get_path_to_root(v_start)

# Find the Lowest Common Ancestor (where the paths merge)
common_node = None
for node in path_v:
    if node in path_u:
        common_node = node
        break

# Trace path from V up to Common Node
curr = v_start
while curr != common_node:
    par, edge_id = tree_parents[curr]
    # Check direction relative to loop
    edge = pipes[edge_id]
    # If edge goes par->curr, it matches our trace direction
    if edge['u'] == curr and edge['v'] == par:
        d = 1
    else:
        d = -1
    this_loop.append((edge_id, d))
    curr = par

# Trace path from U up to Common Node (this is the other side)
u_side = []
curr = u_start
while curr != common_node:
    par, edge_id = tree_parents[curr]
    edge = pipes[edge_id]
    # If edge goes par->curr, it matches trace
```

```
            if edge['u'] == par and edge['v'] == curr:
                d = 1
            else:
                d = -1
            u_side.append((edge_id, d))
            curr = par

        # Add the U side in reverse to close the loop
        for item in reversed(u_side):
            this_loop.append(item)

        loops.append(this_loop)

print(f"Found {len(loops)} loops. Ready to solve.")

# 5. HARDY CROSS SOLVER
# Final Step to Get the DIscharge value
print("Step 4: Starting Iterations...")

iteration = 0
while True:
    iteration += 1
    max_correction = 0.0

    # Go through every loop
    for loop_data in loops:
        sum_H = 0.0      # Numerator: Sum(r * Q * |Q|)
        sum_2rQ = 0.0    # Denominator: Sum(2 * r * |Q|)

        # Calculate sums
        for pid, direction in loop_data:
            p = pipes[pid]
            q = p['Q']
            r = p['R']

            # Flow relative to loop direction
            q_loop = q * direction

            sum_H += r * q_loop * abs(q_loop)
```

```
            sum_2rQ += 2 * r * abs(q_loop)

        # Calculate Delta Q
        if sum_2rQ == 0:
            delta_Q = 0
        else:
            delta_Q = -sum_H / sum_2rQ

        # Apply correction to all pipes in this loop
        for pid, direction in loop_data:
            pipes[pid]['Q'] += delta_Q * direction

        # Keep track of biggest change
        if abs(delta_Q) > max_correction:
            max_correction = abs(delta_Q)

    # Check if we are done
    if max_correction < ERROR_LIMIT:
        print(f"Converged! Finished in {iteration} iterations.")
        break

    if iteration >= MAX_ITERS:
        print("Stopped: Too many iterations.")
        break

    if iteration % 50 == 0:
        print(f"Iteration {iteration}: max error = {max_correction:.5f}")

# Print the Output for our Problem
print("\nFINAL RESULTS TABLE")
print("-" * 65)
print(f"{'Pipe':<5} | {'Nodes':<8} | {'Length':<8} | {'Dia':<6} | {'Flow (m3/s)':<12} | {'Direction'}")
print("-" * 65)

for pid in sorted(pipes.keys()):
    p = pipes[pid]
    q = p['Q']
    # Making the direction string readable
    if q >= 0:
```

```
        dir_str = f"{p['u']} -> {p['v']}"
        display_q = q
    else:
        dir_str = f"{p['v']} -> {p['u']}"
        display_q = -q
print(f"{p['id']:<5} | {p['u']}-{p['v']:<6} | {p['L']:<8} | {p['D']:<6} | {display_q:<12.4f} | {dir_str}")
print("-" * 65)
```

Flowchart of the Code:-



Output and Table Containing Flow in Every Pipe :-

```
Step 1: Calculating Resistance (R) values for each Pipes...
Step 2: Assigning initial flow to path A -> G...
Initial flow assigned.
Step 3: Finding fundamental loops required for error calcutions ...
Found 14 loops. Ready to solve.
Step 4: Starting Iterations...
Iteration 50: max error = 0.00216
Converged! Finished in 57 iterations.

FINAL RESULTS TABLE
-----------------------------------------------------------------
Pipe  | Nodes    | Length   | Dia    | Flow (m3/s)  | Direction
-----------------------------------------------------------------
1     | A-B      | 300      | 0.4    | 27.0321      | A -> B
2     | B-C      | 250      | 0.35   | 2.6713       | B -> C
3     | C-D      | 200      | 0.3    | 0.2041       | D -> C
4     | D-A      | 280      | 0.32   | 14.9561      | A -> D
5     | B-E      | 220      | 0.28   | 1.0073       | E -> B
6     | E-C      | 260      | 0.3    | 1.9994       | E -> C
7     | C-F      | 240      | 0.34   | 2.6369       | F -> C
8     | F-D      | 210      | 0.33   | 2.5333       | F -> D
9     | E-F      | 230      | 0.26   | 0.7080       | E -> F
10    | A-E      | 270      | 0.36   | 21.2271      | A -> E
11    | B-F      | 290      | 0.31   | 0.4905       | F -> B
12    | D-E      | 250      | 0.29   | 1.8210       | E -> D
13    | C-A      | 310      | 0.38   | 23.5968      | A -> C
14    | F-A      | 280      | 0.34   | 17.6696      | A -> F
15    | E-G      | 260      | 0.27   | 15.6914      | E -> G
16    | G-C      | 300      | 0.35   | 31.1084      | C -> G
17    | G-D      | 320      | 0.3    | 19.1063      | D -> G
18    | F-G      | 240      | 0.25   | 12.7169      | F -> G
19    | B-G      | 310      | 0.33   | 25.8586      | B -> G
20    | A-G      | 350      | 0.37   | 39.5183      | A -> G
-----------------------------------------------------------------
```

Literature Review: Evolution and Application of Pipe Network Analysis

The analysis of flow distribution in hydraulic networks has evolved significantly from manual iterative procedures to advanced computational algorithms. This review examines five seminal and contemporary research papers that establish the theoretical framework and modern application of the Hardy Cross method.

1. Analysis of Flow in Networks of Conduits or Conductors

- Author(s): Hardy Cross
- Year: 1936
- Significance: Foundational Paper
- Abstract: In this seminal work, Cross proposed the first practical engineering method for solving complex network flows where the relationship between head loss and flow is non-linear. He introduced a method of successive corrections (balancing heads) to satisfy the conservation of mass at nodes and energy around loops. This paper laid the groundwork for all modern hydraulic analysis.
- Link: University of Illinois Repository (PDF)

2. Implementation of the Hardy-Cross Method for the Solution of Piping Networks

- Author(s): A.M.G. Lopes
- Year: 2004
- Significance: Computational Implementation
- Abstract: This study bridges the gap between theory and digital application. Lopes developed a generalized computer algorithm capable of solving networks with arbitrary topology. The paper demonstrates that automating the Hardy Cross method ensures strict adherence to energy conservation laws along every closed path, which is computationally prohibitive to verify manually for large systems.
- Link: CORE Repository (PDF)

3. Short Overview of Early Developments of the Hardy Cross Type Methods

- Author(s): Dejan Brkić & Pavel Praks
- Year: 2019
- Significance: Historical & Comparative Analysis
- Abstract: This paper provides a critical review of the method's development, comparing Cross's original "balancing of heads" approach with the "balancing of flows" method. It concludes that despite the emergence of matrix-based solvers like Newton-Raphson, the Hardy Cross method remains mathematically robust and pedagogically essential for understanding hydraulic equilibrium.
- Link: MDPI Applied Sciences (Full Text)

4. Modelling of Hardy Cross Method for Pipe Networks

- Author(s): Pankaj Dumka, Nitin Samaiya, Sumit Gandhi, & Dhananjay R. Mishra
- Year: 2023
- Significance: Modern Python Automation
- Abstract: This recent study focuses on eliminating human error in hydraulic design by automating the Hardy Cross iterations using Python. The authors validated their Python model against four complex network benchmarks, proving that modern scripting languages can effectively handle the iterative load of the Hardy Cross method with high precision.
- Link: International Journal of Mechanical Engineering (PDF)

5. Comparison of Newton Raphson, Linear Theory, and Hardy Cross Methods

- Author(s): Mujeeb Safi
- Year: 2024
- Significance: Performance Benchmarking
- Abstract: Safi conducted a comparative analysis of three dominant numerical methods for looped water supply networks. The study concluded that while the Newton-Raphson method offers faster convergence rates, the Hardy Cross method yields results of comparable accuracy and remains a highly reliable tool for the planning and analysis of gravity-main systems where computational stability is a priority.
- Link: Semantic Scholar (Abstract & Access)